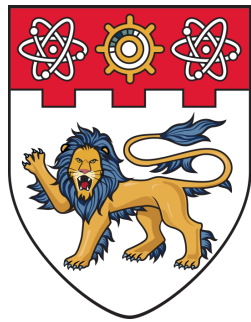


**ZERO-SHOT OBJECT DETECTION AND REFERRING EXPRESSION
COMPREHENSION USING VISION-LANGUAGE MODELS**



**NANYANG
TECHNOLOGICAL
UNIVERSITY**

SINGAPORE

A MANICKA PRAVEEN

U2021866D

SCHOOL OF MECHANICAL AND AEROSPACE ENGINEERING

NANYANG TECHNOLOGICAL UNIVERSITY

Year 2023/2024

ZERO-SHOT OBJECT DETECTION AND REFERRING EXPRESSION
COMPREHENSION USING VISION-LANGUAGE MODELS

Submitted by
A Manicka Praveen
U2021866D

Under the supervision of
Assoc. Prof. Ang Wei Tech

School of Mechanical and Aerospace Engineering

A final year project report presented to
Nanyang Technological University
in partial fulfilment of the requirements of the
Degree of Bachelor of Engineering (Mechanical Engineering)

Year 2023/2024

Abstract

This project focused on constructing a comprehensive perception pipeline integrating Natural Language Processing (NLP), zero-shot object detection, and Referring Expression Comprehension (ReC) within a ROS (Robot Operating System) framework. The aim was to enhance robotic assistive devices in accurately interpreting natural language commands and grounding language to physical objects in the real world. To achieve this, we compared various combinations of zero-shot object detectors and ReC models, specifically OWL-ViT and Grounding DINO for zero-shot object detection; and ReCLIP and GPT-4 for ReC. Our evaluation assessed the models' capabilities in counting, spatial reasoning, understanding superlatives, handling multiple instances, self-referential comprehension, and identifying household objects. The findings were showed that GPT-4 outperformed ReCLIP as for the purpose of ReC, and the combination of Grounding DINO and GPT-4 proved to be the best zero-shot object detector and ReC pair.

Acknowledgement

I would like to express my deepest gratitude to Assoc Prof Ang Wei Tech for his guidance and mentorship. His profound knowledge and insights have significantly contributed to the success of this project and have been a great source of inspiration to me.

I am also profoundly thankful to J-Anne Yow, Graduate Student Research Assistant, for her unwavering support and constant guidance throughout my final year project. Her expertise, patience, and encouragement have been invaluable to my research and personal growth.

I must also acknowledge the Rehabilitation Research Institute of Singapore for providing me with the necessary resources and an enriching environment to conduct my research. The facilities and support from the institute have been crucial in the completion of my project.

My heartfelt appreciation extends to my family and friends for their endless love, encouragement, and support. Their belief in me and constant encouragement have been my strength and motivation throughout this journey.

This project would not have been possible without the collective support and guidance from all mentioned above. I am eternally grateful for the opportunity to learn from such distinguished individuals and institutions.

Contents

1	Introduction	1
1.1	Research Motivation	1
1.2	Project Objectives and Scope	2
1.3	Organization	2
2	Literature Review	4
2.1	Object Detection	4
2.1.1	Supervised Object Detection	5
2.1.2	Zero-Shot Object Detection	7
2.2	Referring Expression Comprehension	10
3	Methodology	14
3.1	Zero-Shot Object Detection	14
3.2	Natural Language Processing	14
3.3	Referring Expression Comprehension	17
3.3.1	ReCLIP	17
3.3.2	GPT-4	17
3.4	Proposed Architecture	19
3.5	ROS Integration	20
4	Experiments	22
4.1	Experiment Dataset	22
4.2	Implementation Details	24
4.2.1	ReCLIP	24
4.2.2	GPT-4	25
4.2.3	Grounding DINO	25
4.2.4	OWL-ViT	26
4.3	Evaluation Framework	27

5	Results	28
5.1	Analysis	30
6	Conclusion and Future Work	35

List of Figures

1.1	An example showing Mask R-CNN model’s ability to only detect a limited number of images	1
2.1	Architecture of CLIP	7
2.2	Architecture of OWL-ViT	8
2.3	Grounding DINO architecture	9
2.4	Grounding DINO architecture	10
2.5	Grounding DINO architecture	11
2.6	ReCLIP architecture	12
2.7	GPT-4V Scene Understanding	13
2.8	GPT-4V Counting ability	13
2.9	GPT-4V Localisation inaccuracy	13
3.1	Comparison of Grounding DINO and OWL-ViT	15
3.2	Extract_nouns function	16
3.3	Noun_chunking function	16
3.4	Simple ReCLIP tasks	17
3.5	Spatial reasoning ReCLIP task	18
3.6	Proposed GPT-4 method for ReC	18
3.7	Proposed software architecture using ReCLIP	19
3.8	ROS Software Architecture	21
4.1	Example of images from the COCO Dataset	22
4.2	Examples of Image-Text pairs used to test counting, spatial reasoning, superlatives, multiple proposals, self-referential, and household objects	23
5.1	Grounding DINO successful detection	30
5.2	OWL-ViT failed detection	31
5.3	Expected ReC output	31
5.4	Examples of results for counting (top row), spatial reasoning (middle row), & superlatives (bottom row)	32
5.5	Examples of results for multiple instances (top row), self-referential (middle row), & household objects (bottom row)	33

List of Tables

5.1	Experiment Results	28
-----	------------------------------	----

Chapter 1

Introduction

1.1 Research Motivation

The advancement of robots in assistive daily living (ADL) tasks is critical for enhancing human capabilities through technology. These devices are designed to perform a wide range of ADL tasks, aiming to blur the lines between human abilities and technological assistance. Central to achieving this objective is equipping robots with advanced perception capabilities, enabling them not only to recognize objects but also to understand their context and relevance within dynamic environments.

The current perception pipeline used in Rehabilitation Research Institute of Singapore (RRIS) utilizes the Mask R-CNN framework for object detection, which identifies objects with high precision. However, this approach is limited to recognizing only objects predefined in the MS COCO dataset as shown in Figure 1.1 [1]. It also lacks the functionality for referring expression comprehension (ReC). This limitation hinders the system’s ability to differentiate among multiple objects of the same class in a single frame.



Figure 1.1: An example showing Mask R-CNN model’s ability to only detect a limited number of images

To address these challenges, this project explores the integration of Vision Language Models (VLMs) to achieve zero-shot object detection and enhance ReC capabilities. This innovative ap-

proach has the potential to significantly expand the scope of detectable objects without the need for retraining and improve the system’s ability to understand and interact with its environment, thereby advancing the field of assistive technologies and facilitating more effective human-robot collaboration. This machine learning pipeline can subsequently be integrated with a robotic arm system, allowing the arm to perceive the environment and enable the robot to perform different tasks based on natural language commands.

1.2 Project Objectives and Scope

The objectives of this project are:

1. To enhance the current perception pipeline in order to detect a broader array of objects without requiring prior training.
2. To refine the ability to disambiguate objects of the same class through the implementation of referring expression comprehension (ReC)

The scope of this project is:

1. To explore and benchmark various zero-shot object detection models
2. To investigate and evaluate different ReC methods to effectively disambiguate objects of the same class.
3. To develop and enhance methods for spatial reasoning, allowing for more accurate interpretation and interaction with the detected objects.
4. To integrate zero-shot object detection with ReC methods, creating a comprehensive object detection pipeline that leverages language for improved accuracy and efficiency.

1.3 Organization

Chapter 1 introduces the project, starting with the Research Motivation which highlights the critical role of advanced robot perception for assistive daily living tasks and identifies the limitations of current perception pipelines in object detection and ReC. It lays out the Project Objectives and Scope with the aim to enhance the perception pipeline for broader object detection and refined ReC.

Chapter 2 discusses various zero-shot object detection methods and evaluates the capabilities of different ReC models. The aim here is to develop an integrated model with improved spatial reasoning, ultimately facilitating user interaction with a robotic arm system via natural language commands for household object manipulation.

Chapter 3 elaborates on the process of selecting zero-shot object detection models and the integration of NLP for refining the ReC component within the software architecture. It also covers the proposed software architecture and its integration with ROS, providing a comprehensive overview of the technical framework and tools employed.

Chapter 4 describes the curated experiment dataset designed to test and challenge the ReC models. Implementation details for ReCLIP, GPT-4, Grounding DINO, and OWL-ViT are also detailed, followed by an Evaluation Framework that outlines the methodology for testing the effectiveness of the model combinations.

Chapter 5 presents the results of the experimentation, including a table summarizing the experiment results across different categories such as counting, spatial, and more. An Analysis section follows, interpreting the results and discussing their implications for the performance of the various model pairings in complex referential.

Chapter 6 concludes the report with a wrap-up of the findings and proposes directions for Future Work, suggesting areas for further research and potential enhancements to the current perception pipeline.

Chapter 2

Literature Review

In this paper, we plan to compare and evaluate various zero-shot object detection methods as well as evaluate various referring expression comprehension models' capabilities. Through this, we plan to create a robust integrated zero-shot object detection model which would have a much more intuitive spatial reasoning system as compared to the rule-based approach taken by other models. Following which, this model would be used with a robotic arm system, enabling users to interact with the robot and execute the task of picking up household objects through straightforward natural language commands.

In this study, we plan to develop a zero-shot object detection model with the capability of referring expression comprehension through the use of vision-language models.

2.1 Object Detection

Object detection [2] methods are broadly categorized into two main frameworks: region proposal-based and regression/classification-based. Region proposal-based methods, like R-CNN [3] and its variants, first generate candidate object locations in an image and then classify each location. In contrast, regression/classification-based methods, such as YOLO (You Only Look Once) [4], SSD (Single Shot MultiBox Detector) [5], and Zero-Shot Object Detection [6] predict object categories and locations in a single forward pass, enhancing speed and efficiency. Notably, zero-shot object detection extends these frameworks by recognizing objects not seen during training, leveraging semantic relationships and transfer learning to identify novel categories without explicit examples [6].

Traditional object detection generally focused on classifying different images, and precisely estimating the locations of objects contained in each image [7]. This would mean that models would have to be trained on each new object class that it should identify and classify. Traditional object detection models do not naturally understand where objects are in relation to each other when they

are learning. This poses a problem as we are not able to identify objects using everyday language commands easily.

2.1.1 Supervised Object Detection

The domain of object detection has seen significant advancements in recent years and is expected to play an increasingly pivotal role as technology progresses over the next few decades. Currently, numerous algorithms are utilized for supervised object detection, broadly categorized into two types: one-stage detectors and two-stage detectors. Two-stage detectors first identify approximate areas where objects are present using deep learning methods before classifying these objects, effectively labelling what has been detected. This step also refines the bounding boxes around the presumed location of objects. Conversely, one-stage detectors directly generate object bounding boxes without requiring the preliminary step of identifying regions, offering a quicker and more efficient process suitable for real-time usage [8]. In the discussion that follows, we will explore the key algorithms that fall into these two categories.

Two-Stage Object Detection

Two-stage detection algorithms are more complex compared to single-step detection methods. They operate in two phases: initially, they conduct preliminary analyses to identify regions of interest (RoIs). Following this, the second phase involves classifying these regions and accurately pinpointing the object's location based on the RoIs established in the initial phase. The primary algorithms that utilize this two-stage approach include Faster R-CNN, R-FCN, FPN, and Cascade R-CNN [9].

- **Faster R-CNN:** This algorithm employs R-CNN for generating classification regions as well as for detecting objects. It features a Region Proposal Network (RPN) that shares convolutional layers with the object detection network, significantly reducing the time and computational cost of generating proposals. Faster R-CNN is noted for its enhanced accuracy in detecting objects compared to the original CNN model [10].
- **R-DFPN:** In the realm of object detection, feature pyramids have traditionally been utilized. However, their application has diminished due to the high computational demands. The Feature Pyramid Network (FPN), particularly when integrated into Faster R-CNN frameworks,

has demonstrated notable improvements in performance, offering a more efficient use of multi-layered algorithmic approaches [11].

- **Cascade R-CNN:** The Intersection Over Union (IoU) metric is crucial in object detection for determining the accuracy of a detection as either a true or false positive. The standard IoU threshold, often set at 0.5, can result in frequent misclassifications. To address this, the Cascade R-CNN architecture introduces a multi-stage process with progressively higher IoU thresholds. Each stage of the cascade builds upon the detections of the previous one for training, leading to significantly improved detection performance [11].

One-Stage Object Detection

One-stage detectors are streamlined algorithms designed for object detection in a single step, eliminating the need for an initial phase to identify object regions. This efficiency makes them ideal for real-time detection tasks due to their reduced processing time.

Prominent algorithms in the one-step detection category include YOLO, SSD, RetinaNet, YOLOv3, YOLOv4, and YOLOR [8]. This section will focus on the SSD (Single Shot Detector) algorithm. The SSD is a unified detection algorithm that employs a single neural network to analyze the image. It divides the image into a grid and assigns predefined bounding boxes to each grid location on the feature map. During the prediction phase, it evaluates the likelihood of an object's presence within each box and adjusts the boxes to better fit the object's contours. Unlike some algorithms, SSD does not rely on proposal generation or pixel resampling, making all its computations straightforward and contained within the network, facilitating its integration into various detection systems [12].

While the YOLO (You Only Look Once) algorithm may not be the most precise in object detection, its exceptional speed has made it the preferred choice in many object detection applications. YOLO operates with a depth of 12 layers and has been trained across more than 80 different object categories [5], necessitating substantial computational power and extensive data sets. The YOLO method applies a classification approach that unfolds in two stages: initially identifying the Region of Interest (ROI), followed by the application of a specific neural network, often a fully connected CNN, that manages structures within a uniform grid through parameter sharing. This feature of

CNNs—learning a set of weights for specific locations—contributes to their effectiveness in object detection. The YOLO model initiates detection by partitioning the input image into a grid of cells, with each cell being crucial for the detection task. These cells are tasked with defining boundary boxes and classes, with each box prediction encompassing five elements: height, width, x and y coordinates, and confidence level. The x and y coordinates pinpoint the center of the box relative to the grid cell’s boundaries, whereas the height and width are proportional to the dimensions of the overall image. The confidence prediction correlates with the likelihood, represented on a probability map [13].

2.1.2 Zero-Shot Object Detection

Zero-shot object detection typically leverages substantial vision-language pre-training models like CLIP (Contrastive Language–Image Pre-training) [14]. In contrast to traditional object detection techniques that recognize and categorize only the objects from their training classes, zero-shot detection extends the capability to objects outside the training set [6].

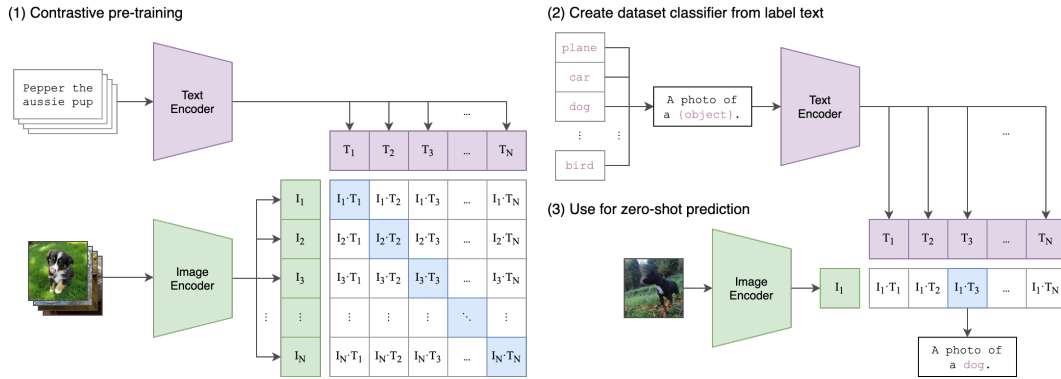


Figure 2.1: Architecture of CLIP

Illustrated in Figure 2.1 [14], conventional image recognition models train both an image feature extractor and a linear classifier to assign labels, whereas CLIP develops an image encoder and a text encoder aimed at identifying correct (image, text) combinations during training. At the operational phase, the model employs the trained text encoder to generate a zero-shot linear classifier, utilizing the names or descriptions of unseen class categories.

Another model facilitating zero-shot object detection is OWL-ViT, which adopts an open-vocabulary detection approach. OWL-ViT can identify objects through any textual queries without

necessitating re-training on labeled data, positioning it as an open-vocabulary detector.

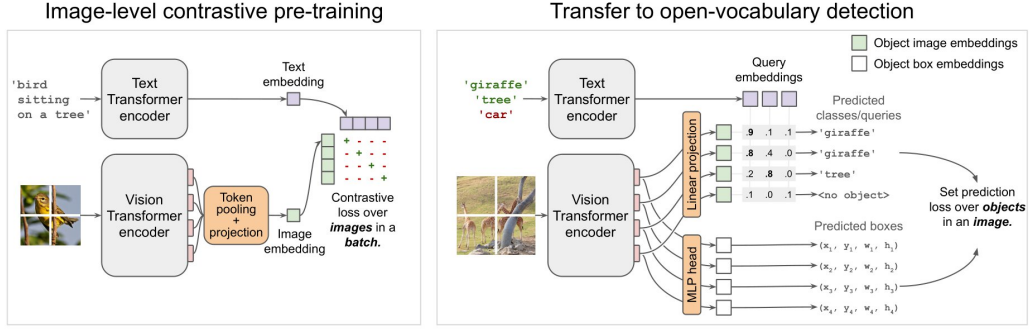


Figure 2.2: Architecture of OWL-ViT

As depicted in Figure 2.2 [15], OWL-ViT initially trains an image and text encoder with image-text pairs in a manner akin to CLIP [14]. Subsequently, these pre-trained encoders are adapted for open-vocabulary object detection by eliminating token pooling and integrating lightweight object classification and localization heads with the outputs of the image encoder. During detection, queries processed by the text encoder are utilized for classifying objects, and the model undergoes further refinement on standard object detection datasets. At inference, it uses text-based embeddings for detecting a wide range of objects or image-based embeddings for few-shot detection, which relies on minimal example input.

A notable model in zero-shot object detection is Grounding DINO [16]. This model utilizes a self-supervised learning strategy by integrating DINO (DETR with Improved deNoising anchor boxes) [17] with GLIP’s grounded pre-training approach [18]. The DINO framework, based on a transformer architecture, is designed for robust object detection and executes end-to-end optimization without relying on manually designed components. Conversely, GLIP is tailored for phrase grounding, a process that connects textual phrases to their corresponding visual elements in images or videos, thus facilitating the association of text with visual data.

The architecture of Grounding DINO is crafted to seamlessly integrate visual and linguistic information. It incorporates a dual-stream framework as depicted in Figure 2.3, where a visual backbone such as the Swin Transformer [19] extracts multi-scale image features, and a linguistic backbone like the BERT model [20] processes the text data.

The combined outputs from these streams are enhanced by a feature enhancer, transforming

them into a cohesive representation space. This enhancement process employs deformable self-attention mechanisms for the visual features and standard self-attention for the textual features, as illustrated in Figure 2.4.

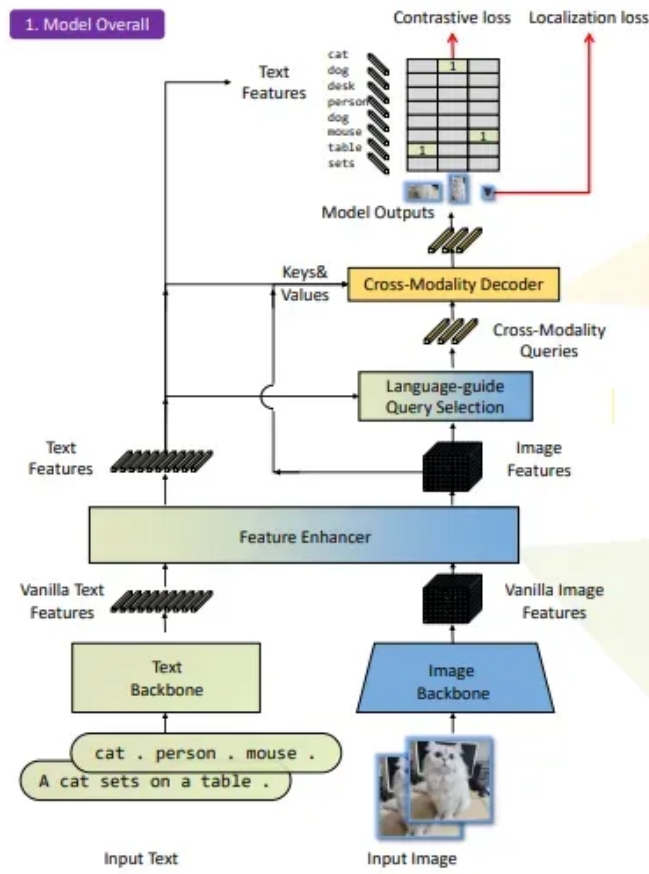


Figure 2.3: Grounding DINO architecture

The purpose of Grounding DINO is to identify and label objects in images based on textual descriptions provided as input. This objective is facilitated by a language-guided query selection mechanism shown in Figure 2.5, which selects the most pertinent features from both text and visual inputs to assist the decoder in pinpointing object locations and attaching accurate labels based on the associated textual descriptions.

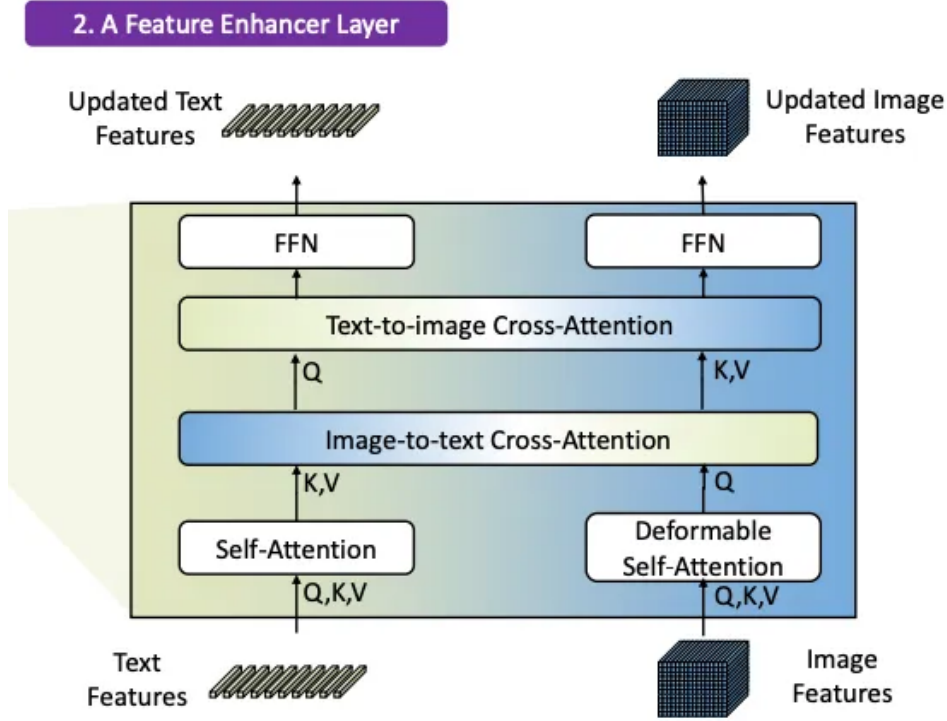


Figure 2.4: Grounding DINO architecture

2.2 Referring Expression Comprehension

In the ReC task, a model is provided with an image alongside a textual expression describing an entity within that image. The objective is to identify the object (via bounding box) that aligns most closely with the given expression. One approach used in ReC is Colorful Prompt Tuning (CPT) [21], where proposal regions are color-coded and a masked language prompt that appends "in [MASK] color" to the referring expression is employed. The most likely color predicted by the pre-trained masked language model (MLM), VinVL [22], is selected to determine the correct region.

Another strategy employed for ReC involves gradient-based visualizations, a cluster of methods that identifies which image segments are pivotal for a model’s decision-making across various computer vision applications. A prominent method within this category is GradCAM [23], which enhances the interpretability of convolutional neural networks (CNNs) by providing visual justifications for their predictive choices, thus clarifying which parts of an image contribute to its classification.

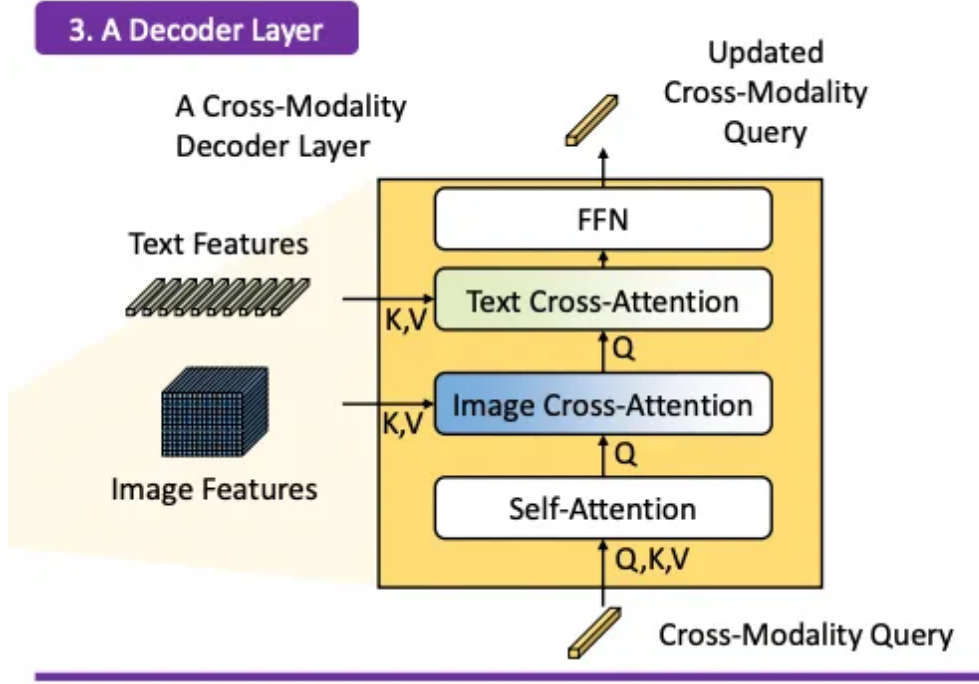


Figure 2.5: Grounding DINO architecture

To integrate the utilities of zero-shot object detection with ReC, the ReCLIP model [24] was developed. Illustrated in Figure 2.6, ReCLIP comprises two principal mechanisms: scoring object proposals via CLIP and managing spatial relationships among objects. Its Isolated Proposal Scoring (IPS) simplifies ReC to the contrastive pre-training regimen utilized by CLIP, by isolating proposals through image cropping and blurring, subsequently scoring them with CLIP against the descriptions. A parser decomposes the expression into noun chunks, with CLIP generating a proposal likelihood for each chunk. These probabilities, combined with spatial relations parsed from the expressions, are integrated by a spatial relation resolver to pinpoint the optimal proposal. For instance, in the depicted scenario, although CLIP individually rates the cropped image of a cat highest, the relation resolver correctly identifies the actual target, the black and white cat, utilizing relational cues.

Despite these innovations, an experiment to test CLIP’s efficacy in interpreting spatial relations revealed its limitations, with a mere 50% accuracy rate in zero-shot spatial tasks [24]. For example, when presented with a scene description such as ‘the banana to the left of the apple’, CLIP struggled to correctly associate the spatial terms with the appropriate entities. This suggests that while CLIP

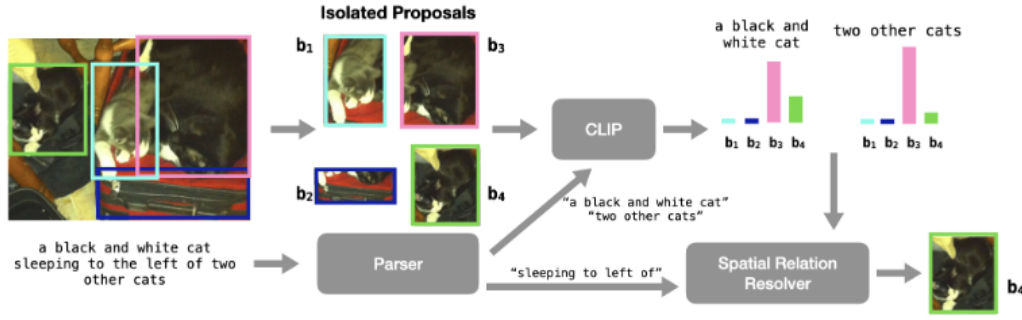


Figure 2.6: ReCLIP architecture

excels in matching image-text pairs, it poorly handles spatial descriptors.

ReCLIP attempts to bolster CLIP’s spatial reasoning with tailored spatial relation heuristics, aiming to enhance its performance in comprehending and applying spatial relationships within scenes [24].

GPT-4V(ision) [25] was also explored in this project. It was identified that GPT-4V demonstrates excellent spatial relationship understanding to identify relations between objects in a scene as seen in Figure 2.7. It also has the ability to count the number of objects in the scene to relatively high accuracy, as shown in Figure 2.8. Both are areas that ReCLIP had difficulty with. However, GPT-4V had difficulties with object localisation where GPT-4V can generate bounding box coordinates in textual format but with major inaccuracies, as shown in Figure 2.9. Although localisation results are more promising in simpler and less cluttered backgrounds, they were still relatively weak.

Concurrent to this Final Year Project, [26] explored the used of Large Language Models (LLMs) for ReC. They proposed ChatRef, which would pass the recognized objects and their attributes (category, location, and color) in a scene as text to an LLM for reasoning. The LLMs GPT-3.5 and GPT-4 were the selected models. Given the textual attribute tuples only, object category, color, center location, size, GPT-3.5 performs unstably on understanding spatial relationships even with heavy prompt engineering, whereas GPT-4 was also able to take in an image input. Using GPT-4 shows strong and stable zero-shot relation reasoning [26].

Prompt:

What is the spatial relation between the frisbee and the man?



GPT-4V:

Based on the image provided, the spatial relation between the frisbee and the man is that the frisbee is above and slightly in front of the man. It also appears to be moving away from the man, as if it has been thrown or released by him.

Figure 2.7: GPT-4V Scene Understanding

Prompt:

Count the number of apples in the image.



GPT-4V:

There are 7 apples in the image.

Prompt:

Count the number of apples in the image.



GPT-4V:

There are 16 apples in the image.

Prompt:

Count the number of oranges in the image.



GPT-4V:

There are 3 oranges in the image.

Prompt:

Count the number of people in the image.



GPT-4V:

There are 3 people in the image.

Figure 2.8: GPT-4V Counting ability

Prompt:

Localize each person in the image using bounding box. The image size is (width, height) = (998, 786)



GPT-4V:

There is only one person in the image, so the bounding box for this person would be:

- Top left: (254, 118)
- Bottom right: (731, 693)

Explanation: The top left point of the bounding box is at (x, y) = (254, 118), which is the top left corner of the person in the image. The bottom right point of the bounding box is at (x, y) = (731, 693), which is the bottom right corner of the person in the image. This bounding box encompasses the entire person in the image.

Visualization of Detected Bbox:



Prompt:

Localize each car in the image using bounding box. What is the image size of the input image?



GPT-4V:

The image size of the input image is 500 x 281 pixels.

Bounding box for the black car:

- Top left corner: (83, 114)
- Bottom right corner: (196, 220)

Bounding box for the white car:

- Top left corner: (287, 96)
- Bottom right corner: (474, 220)

Visualization of Detected Bbox:



Figure 2.9: GPT-4V Localisation inaccuracy

Chapter 3

Methodology

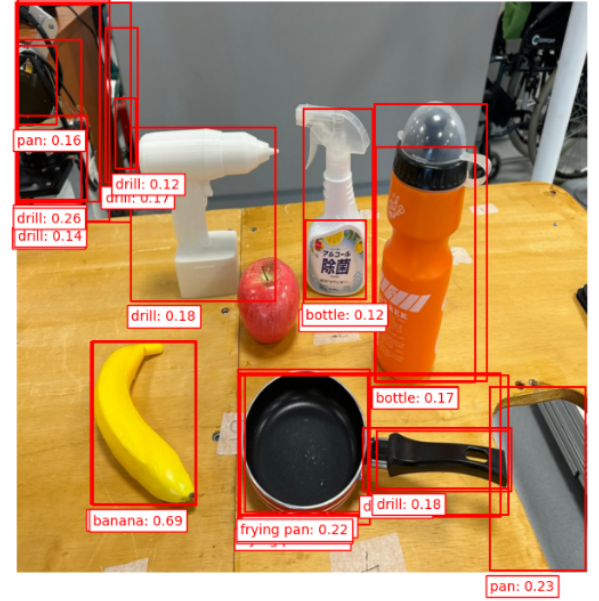
3.1 Zero-Shot Object Detection

Initial comparative testing between two zero-shot object detection models, OWL-ViT and Grounding DINO, revealed distinct differences in performance. Figure 3.1 shows an example of the initial testing, where both models were given the same image and text prompts. Grounding DINO consistently identified and classified target classes with higher precision and confidence levels. OWL-ViT’s results were less reliable with misclassifications, repeated detections of the same object, and numerous false positives. As the purpose of the zero-shot object detection model is to provide the ReC model with the bounding box information of target objects, OWL-ViT’s errors would be very detrimental to the result. Hence, based on these preliminary findings, it was hypothesized that Grounding DINO would outperform OWL-ViT for the purpose of Zero-shot object detection.

For optimal performance, the zero-shot object detectors take in a list of nouns as input. Upon detection of objects in the list, the models output the positions of the objects in the form of bounding box coordinates.

3.2 Natural Language Processing

As mentioned in the previous section, the input for the zero-shot object detectors had to be in the form of a list of nouns. This would mean that a Natural Language Processing (NLP) method would have to be used to extract only the nouns in the given sentence. To perform noun extraction from a sentence utilizing NLP, various approaches can be employed, including rule-based chunking methods like Part-of-Speech (POS) tagging [27] and noun phrase chunking[28]. POS tagging is a process in NLP where each word in a text is assigned a label indicating its part of speech (such as noun, verb, adjective, etc.). For the task of extracting nouns, once POS tagging is applied to a sentence, we can selectively retrieve only the nouns from it. Noun phrase chunking involves



Given prompt: Frying pan, Apple, Banana, Drill, Spray bottle, Bottle, Table

Figure 3.1: Comparison of Grounding DINO and OWL-ViT

identifying and extracting noun phrases from a text. A noun phrase is a group of words that functions collectively as a noun. This process is a form of shallow parsing that breaks down text into more manageable pieces for further analysis, without fully parsing the text into its syntactic structure.

Figure 3.2 shows how to make the *extract_nouns* function designed to take a string input sentence and return a list of nouns identified within that sentence. The process involves several steps, utilizing the nltk library's *word_tokenize* and *pos_tag* functions. The sentence is tokenized into individual words using the *word_tokenize* method, which splits the sentence into components based on word boundaries. The list of words (*words*) is then passed to the *pos_tag* function, which assigns a part-of-speech tag to each word. The resulting tagged_words is a list of tuples where each tuple contains a word and its corresponding part-of-speech tag. The function then iterates over *tagged_words* and constructs a list of nouns. This is done through a list comprehension that checks if the part-of-speech tag starts with 'N', which denotes various forms of nouns (e.g., NN for singular nouns, NNS for plural nouns, NNP for proper singular nouns, etc.). Finally, the function returns the list nouns, which contains all the words from the original sentence that have been classified as

nouns.

```
def extract_nouns(sentence):
    words = word_tokenize(sentence)
    tagged_words = pos_tag(words)
    nouns = [word for word, pos in tagged_words if pos.startswith('N')]
    return nouns
```

Figure 3.2: Extract_nouns function

Another method that was explored for the purpose of noun extraction is the use of the spaCy [28] library. For this task, the *noun_chunking* method as shown in Figure 3.3. The input sentence is parsed to create a *doc* object, representing the linguistic annotations of the text. Parsing is a complex process that assigns lexical attributes and syntactic dependencies to each word. SpaCy's *.noun_chunks* property is utilized to extract noun phrases. Noun chunks are spans of tokens that have a noun as their syntactic head and include the words that modify the noun, providing a more complete semantic unit than standalone nouns. The function collects the *.root.text* of each noun chunk, which corresponds to the central noun that anchors the phrase. The function concludes by returning the *nouns* list, a compilation of the root nouns from the sentence's noun phrases.

```
import spacy
nlp = spacy.load('en_core_web_sm')

def noun_chunking(sentence):
    # Process the sentence to create a SpaCy document
    doc = nlp(sentence)
    # Initialize an empty list to hold the nouns or noun chunks
    nouns = []
    # Iterate over the noun chunks in the document
    for chunk in doc.noun_chunks:
        nouns.append(chunk.root.text)
    return nouns
```

Figure 3.3: Noun_chunking function

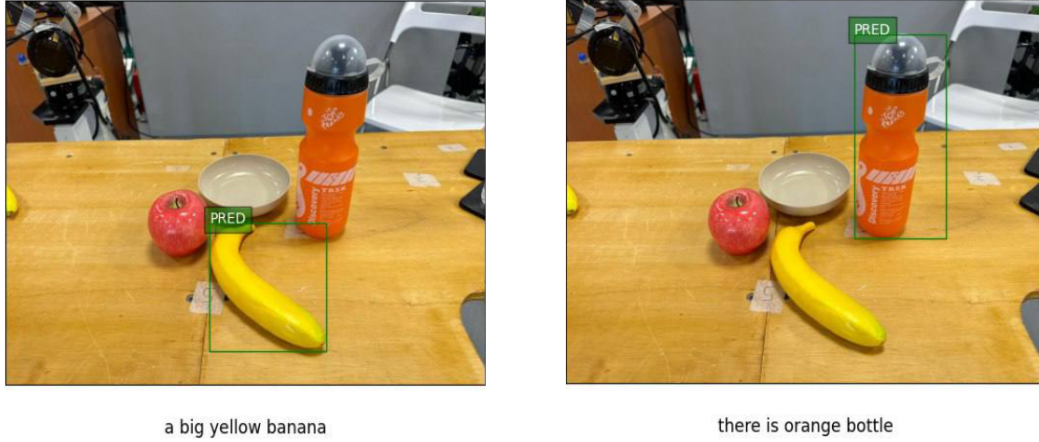


Figure 3.4: Simple ReCLIP tasks

3.3 Referring Expression Comprehension

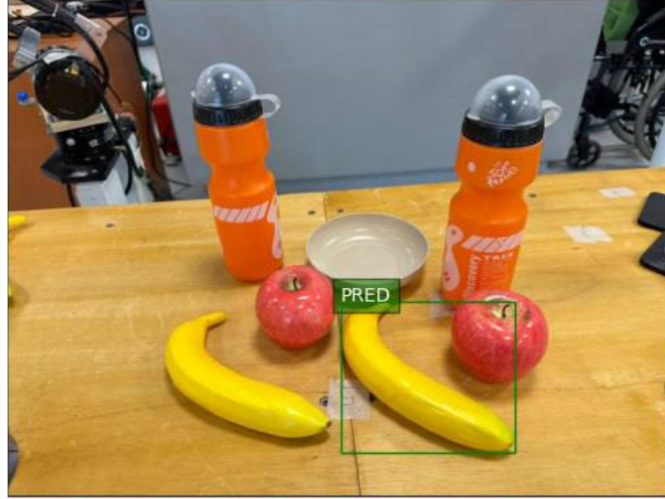
3.3.1 ReCLIP

As ReCLIP is the state-of-the-art method for ReC [24], it was important to conduct initial testings on the ReCLIP model to assess its proficiency for the purpose of ReC. Through testing, it became clear that ReCLIP has the sole purpose of ReC only, and it requires bounding box information of the objects within the scene, the image, and the text prompt for the model to output the bounding box of the target object referred to by the text prompt. This was developed as part of the project, as explain in Section 3.5.

Through the initial testing, it was observed that ReCLIP is able to carry out ReC for simple tasks such as the tasks shown in Figure 3.4. However, if the tasks involved spatial reasoning, counting, or more complex prompts with multiple instances, the model was not very effective. An example is shown in Figure 3.5.

3.3.2 GPT-4

Due to the emergent properties of large language models, we were interested to evaluate GPT-4's ability to perform ReC. The proposed method to use GPT-4 for the purpose of ReC would be to provide the model with bounding box information of all relevant objects in the scene, together with the text prompt and image. This is done to compensate for the models poor object localisation



there is a yellow banana on the left

Figure 3.5: Spatial reasoning ReCLIP task

abilities [26]. The proposed method of using GPT-4 for ReC is shown in Figure 3.6.

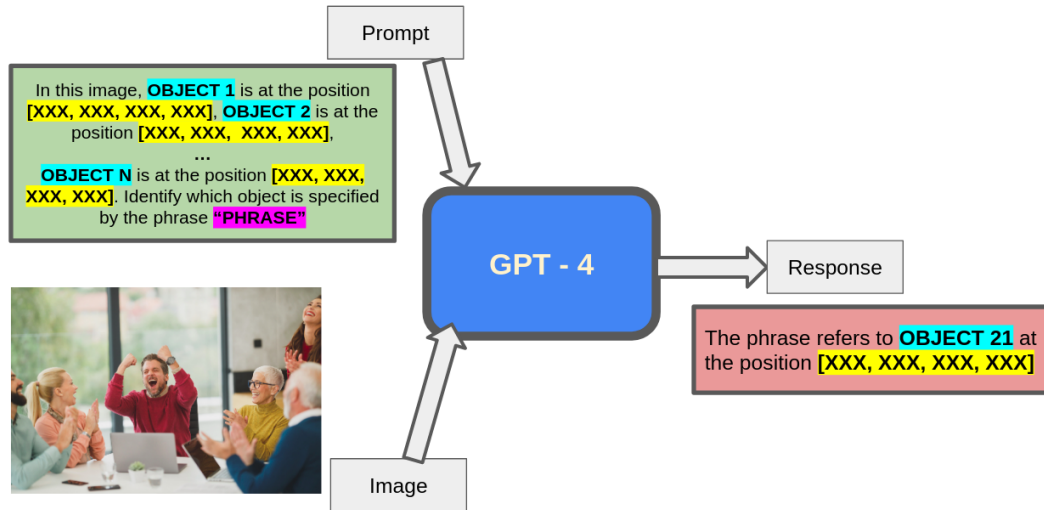


Figure 3.6: Proposed GPT-4 method for ReC

In this method, the locations of all relevant objects will be provided in the prompt to GPT-4 to obtain the location of the object of interest.

3.4 Proposed Architecture

The goal was to create a robust system capable of interpreting natural language commands related to objects within a given image, referred to as ReC tasks. To accomplish this, the architecture needed to process both textual and visual data, extract relevant information, and integrate these insights to accurately identify objects described by the user.

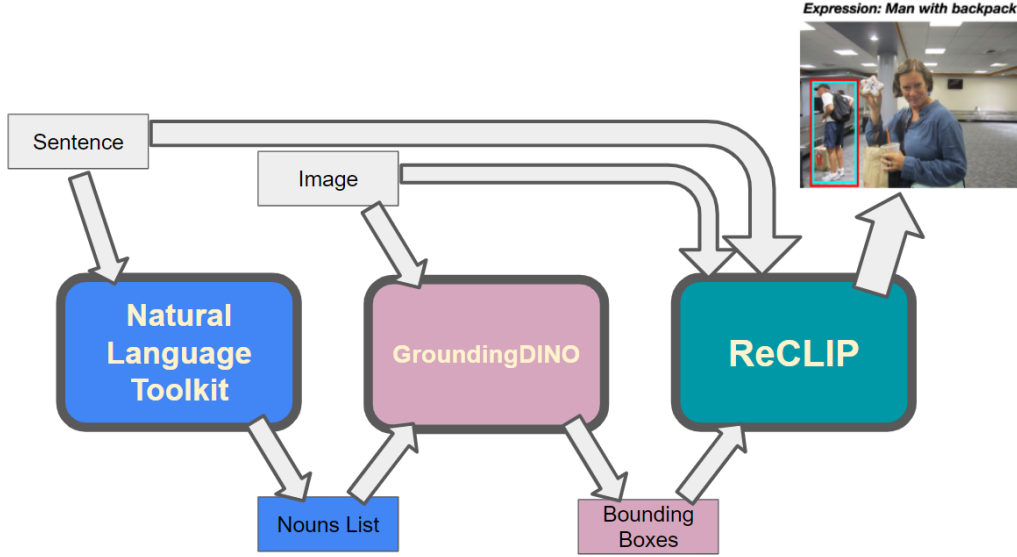


Figure 3.7: Proposed software architecture using ReCLIP

In the process of implementing ReCLIP within our experimental framework, it was observed that the model inherently lacks the capacity for standalone operation. It required supplementary information, specifically bounding boxes identifying all relevant objects within the scene, accompanied by the image and a descriptive sentence. To fulfill this prerequisite, GroundingDINO was selected as the zero-shot object detection framework, demonstrating optimal performance when target object were fed in the form of a list. This requirement prompted the integration of a Natural Language Processing (NLP) package, tasked with extracting nouns from the input sentence, thereby generating the necessary list of objects for GroundingDINO’s operation. For this purpose, the Natural Language Toolkit (NLTK) library [29] was used.

During the initial testing phase of the proposed software architecture, it was identified that ReCLIP exhibited limitations in handling ReC tasks, particularly those requiring an understanding of spatial relations, counting, and interpreting complex sentences involving multiple proposals,

consistent with observations documented in relevant literature. In light of these constraints, an alternative approach was explored, integrating GPT-4 as a substitute for ReCLIP to serve as the ReC component within the software architecture.

Additionally, during the initial evaluations, it was observed that NLTK failed to accurately identify nouns in complex sentence structures. To address this issue, an alternative approach was employed utilizing spaCy’s noun chunking [28] function. This method aimed to enhance the accuracy of noun identification, thereby improving the software architecture’s ability to generate a comprehensive list of objects for subsequent processing steps.

3.5 ROS Integration

Our system architecture leverages the flexibility and modularity of the Robotic Operating System (ROS) framework to manage the complex interactions between its components. Central to this architecture is the Perception Manager, a script responsible for coordinating the sequence of operations required to interpret and act upon user requests. Each user request initiates a pipeline that integrates natural language processing, zero-shot object detection, and referring expression comprehension to accurately identify objects within images based on descriptive sentences.

The process begins when the Perception Manager receives a new request from the user. This request triggers the first operation in the pipeline: the extraction of nouns from the provided sentence, a task achieved through advanced natural language processing techniques. We employed spaCy’s noun chunking capability for this purpose, given its proficiency in accurately identifying and grouping nouns and noun phrases, which represent the potential objects of interest within the image.

Upon isolating the relevant nouns, the Perception Manager forwards these, along with the original image, to the GroundingDINO server. This server is tasked with the zero-shot object detection operation, utilising the list of nouns as a guide to identify and localise the objects within the image. GroundingDINO’s response, consisting of bounding boxes surrounding the detected objects, signifies the completion of the object detection phase.

The subsequent and final phase involves the Perception Manager forming a new request, this time directed at the ReCLIP server. This request encapsulates the user’s original sentence, the

detected objects' bounding boxes provided by GroundingDINO, and the image itself. The role of the ReCLIP server is to interpret the sentence in the context of the provided image and bounding boxes, determining which object within the bounds is being referred to by the user's description.

Upon completion of this analysis, the ReCLIP server returns a response to the Perception Manager, indicating the bounding box of the object identified as the subject of the user's sentence. This outcome represents the culmination of the system's workflow, achieving the objective of referring expression comprehension by accurately identifying and localising the described object within the image.

This architecture embodies a sophisticated integration of natural language processing, zero-shot object detection, and referring expression comprehension within a ROS framework. The Perception Manager plays a pivotal role in orchestrating this complex sequence of operations, ensuring seamless communication between components and the successful execution of tasks.

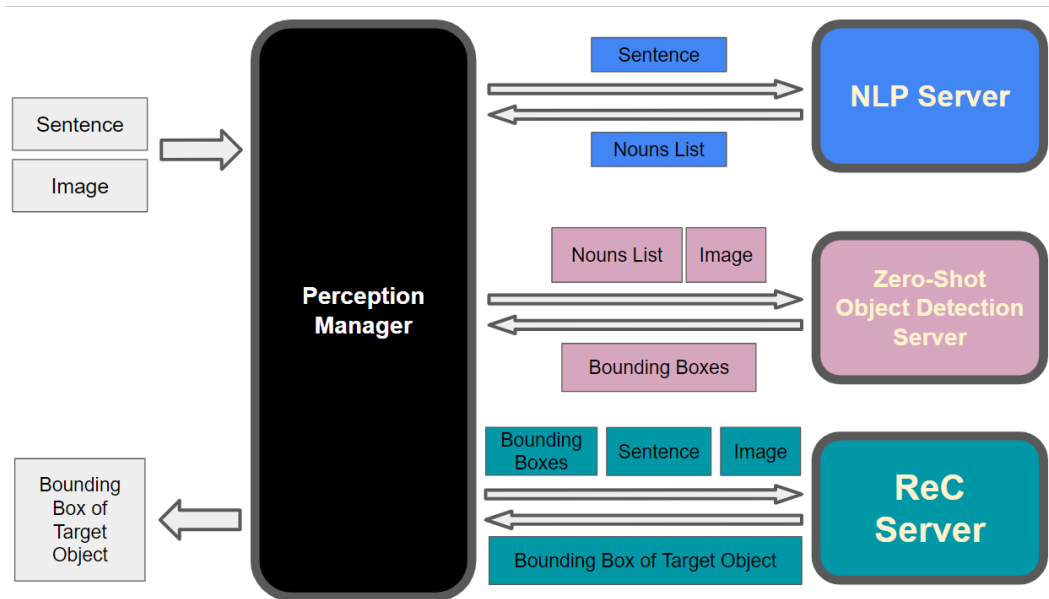


Figure 3.8: ROS Software Architecture

Chapter 4

Experiments

4.1 Experiment Dataset

Initial attempts to utilise random images from the RefCOCO dataset [30] for testing revealed a significant challenge: the inherent complexity of these images often exceeded the scope of straightforward ReC tasks, as shown in Figure 4.1, potentially confounding the accuracy assessments of both ReCLIP and GPT-4.



Figure 4.1: Example of images from the COCO Dataset

To solve this issue and ensure a fair, focused comparison, a curated set of 50 stock images was selected. These test images were carefully chosen to represent the multiple scenarios specifically challenging for ReCLIP, as identified in prior evaluations and documented in the literature. The selected images were categorised into five distinct groups, each embodying a particular challenge area in ReC tasks where ReCLIP’s performance was noted to falter. These categories were as follows:

1. Counting: Images requiring the identification of objects based on numerical queries, testing the models’ abilities to accurately enumerate items within a scene. An example of an image used to test the model’s counting ability is shown in Figure 4.2. Together with the image,

the prompt given was "fifth candle from the left". This would assess the ReC models' ability to handle text prompts which involves counting

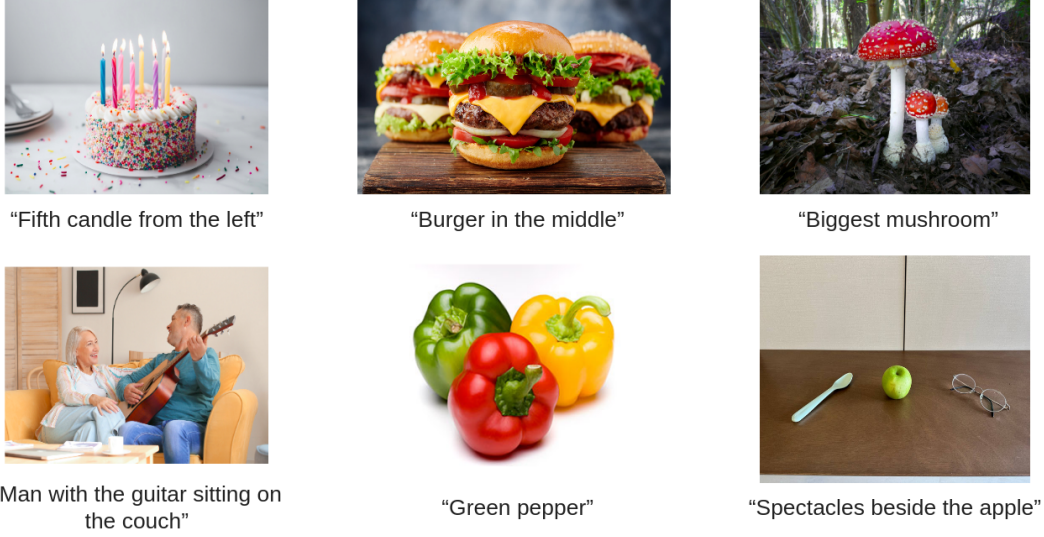


Figure 4.2: Examples of Image-Text pairs used to test counting, spatial reasoning, superlatives, multiple proposals, self-referential, and household objects

2. **Spatial Reasoning:** Scenes necessitating an understanding of spatial relationships among objects, such as "next to", "behind", or "in front of" to identify the referred object correctly. Figure 4.2 shows an example of an image used to test the spatial reasoning of the models. For this image, the text prompt was "burger in the middle". Through this, we aim to see if the model is able to understand the location of each object with respect to one another.
3. **Superlatives:** Images designed to assess the models' capacity to handle superlative expressions (e.g., "the largest," "the smallest") in identifying objects. Figure 4.2 contains an example of an image used to test the models' understanding of superlatives prompts. The prompt given for the image is "biggest mushroom". Depending on how many of these tasks the models completes successfully, we would be able to understand the models' ability to carry out ReC tasks with superlative prompts.
4. **Multiple Proposals:** Scenarios involving several potential subjects of reference, challenging the models' abilities to discern among multiple valid objects based on the given description. Figure 4.2 contains an example of an image used to test this. The prompt given was "man

with the guitar sitting on the couch”. Through this, we would be able to find out the ReC models would be able to handle more complex tasks.

5. **Self-referential (Size/Colour/etc.):** Images that require the models to comprehend self-referential cues, such as colour or size descriptors, to accurately pinpoint the described object. Figure 4.2 shows an example of an image used to assess the models’ self-referential understanding. Together with this image, the prompt given was ”green pepper”. Through these tasks, we will be able to assess the models’ ability to handle self-referential tasks.
6. **Household Objects:** Additionally, images of everyday household objects were also added to the dataset. The purpose of doing so is to assess the model’s capability in performing ReC tasks for our specific use-case. Figure 4.2 contains an example of an image used to test the models’ ability in carrying out tasks related to household objects. The prompt given with this image is ”spectacles beside the apple”. Essentially, this portion of the dataset would be able to give us the best understanding on how accurate our model will be for our use-case as the text prompts were designed to mimic what may be asked by a human user.

4.2 Implementation Details

4.2.1 ReCLIP

- **Model Selection:** The ReCLIP model integrates two CLIP architectures: the ResNet (RN50x16) and the Vision Transformer (ViT-B/32). These models are pre-trained on diverse datasets and are known for their strong performance in vision-language tasks.
- **Box Representation Method:** Configured to use a combination of ’crop’ and ’blur’ techniques, allowing the model to focus on salient regions of an image while maintaining contextual information.
- **Box Method Aggregator:** The ’sum’ aggregator is employed, indicating that the model consolidates insights from multiple representations to inform its predictions.
- **Device:** Execution is set to run on GPU if available (device = ”cuda:0”), but can fall back to CPU (device = ”cpu”) depending on the system’s capabilities and the availability of GPU

resources. As GPU was available, the former was used for our testing purposes.

- **Blur Standard Deviation:** A high standard deviation for the Gaussian blur (e.g., `blur_std_dev` = 100) is used to effectively defocus the background, emphasizing the object of interest.
- **Non-square Resizing:** The flag `non_square_size` is set to `False`, maintaining the aspect ratio of images during resizing operations.
- **Position Embedding Expansion:** The `expand_position_embedding` parameter is set to `False`, indicating a standard approach to position embedding within the model.
- **Temperature:** Set to 1.0 for the softmax calculation, providing a balanced weight to the model outputs.

4.2.2 GPT-4

- **Model Selection:** The "gpt-4-vision-preview" model is used for experiments, optimized for vision-related tasks. The model not only has the ability to understand images, it also contains all other GPT-4 Turbo capabilities. It has been trained on training data up to April 2023.

4.2.3 Grounding DINO

- **Model Architecture:** At its heart, Grounding DINO utilizes a Transformer architecture, enabling the model to capture intricate relationships between visual features and textual descriptors through self-attention mechanisms. This design is pivotal for tasks requiring a deep understanding of context and spatial relationships conveyed in natural language.
- **Model Parameters:**
 - **Detection Threshold (`box_threshold`):** Set to 0.3, this threshold serves as a filter for the detection confidence level. Only detections with confidence scores exceeding this threshold are considered valid, ensuring a focus on high-probability predictions.
 - **Text Threshold (`text_threshold`):** Defined at 0.25, it filters the linguistic predictions to retain only those with sufficient confidence. This parameter is crucial for honing in

on relevant textual descriptions and linking them accurately to detected objects in the image.

- **Preprocessing and Image Transformation:** Input images undergo a series of transformations—resizing, tensor conversion, normalization—tailored to align with the training distribution, enhancing model compatibility with diverse image inputs.
- **Grounding Strategy and Phrase Extraction:** The model employs a sophisticated grounding method, extracting phrases from model outputs based on tokenized captions and position mapping, facilitated by the Transformer’s ability to evaluate the entire input image and descriptive text in a cohesive, context-aware manner.

4.2.4 OWL-ViT

- **Model Architecture:** OWL-ViT leverages the "google/owlvit-base-patch32" variant, known for its robust performance across a wide range of vision tasks. This base model is pre-trained, facilitating immediate application to object detection without the need for further training.
- **Model Integration:** The OwlViTProcessor and OwlViTForObjectDetection are instantiated from the "google/owlvit-base-patch32" checkpoint. This setup prepares the model for processing images and understanding textual queries for object detection.
- **Detection Configuration:**
 - **Image Processing:** Images are loaded and processed using PIL, ensuring they are in the correct format for the model. The processor then combines text prompts with images to create model inputs.
 - **Threshold Setting:** A detection threshold (threshold=0.1) is applied to filter predictions, ensuring that only objects detected with confidence above this threshold are considered. This parameter is crucial for balancing precision and recall in detection tasks.
- **Output Processing:**

- **Post-Processing:** The `post_process_object_detection` method from the processor converts model outputs into a format suitable for analysis and visualization, adhering to the Pascal VOC annotation format.
- **Detection Results Compilation:** Detected objects are categorized with their names and bounding box coordinates, facilitating easy interpretation and subsequent application of detection results.
- **Visualization and Output Handling:** The bounding boxes and labels predicted by OWL-ViT can be visualized on the input image using drawing utilities from PIL, allowing users to visually verify and assess the detection accuracy. Formatted detection results are saved to a designated output directory, ensuring that object detection insights are accessible for further analysis or integration into larger systems.

4.3 Evaluation Framework

To identify the most effective configuration for zero-shot object detection combined with ReC models, an experimental framework was devised containing four distinct model pairings. The zero-shot object detectors being OWL-ViT and Grounding DINO, and the ReC models being ReCLIP and GPT-4. The four possible combinations of zero-shot object detectors and ReC are OWL-ViT + ReCLIP, Grounding DINO + ReCLIP, OWL-ViT + GPT-4. The effectiveness of these combinations was rigorously tested against the experiment dataset specifically designed for this purpose.

This evaluation aimed to not only determine the superior model pairing but also to quantify the impact of the zero-shot object detector performance on the overall efficiency of the perception pipeline. The findings from this assessment are expected to provide clarity on the relationship between zero-shot object detection and ReC models, thereby facilitating a deeper understanding of their integrated capabilities.

Chapter 5

Results

Table 5.1: Experiment Results

Category	OWL-ViT + ReCLIP	Grounding DINO + ReCLIP	OWL-ViT + GPT-4	Grounding DINO + GPT-4
Counting	20%	30%	10%	70%
Spatial	40%	30%	70%	80%
Superlatives	10%	30%	10%	80%
Multiple Instances	50%	70%	60%	100%
Self-referential	10%	40%	30%	60%
Household Objects	30%	30%	80%	100%
Overall Accuracy	26.67%	38.33%	43.33%	81.67%

The assessment of four distinct model pairings — OWL-ViT + ReCLIP, Grounding DINO + ReCLIP, OWL-ViT + GPT-4, and Grounding DINO + GPT-4 — yielded a comprehensive understanding of each configuration’s ability to interpret and process referential expressions within images. The outcomes across various categories of referential comprehension tasks as shown in Table 5.1 are detailed below:

- **Counting:** The capacity for object enumeration was tested, with Grounding DINO + GPT-4 leading at 70% accuracy, highlighting its proficiency in counting within images. OWL-ViT + ReCLIP and Grounding DINO + ReCLIP had comparable outcomes at 20% and 30%, respectively, while OWL-ViT + GPT-4 lagged behind with a 10% success rate.
- **Spatial Reasoning:** Grounding DINO + GPT-4 emerged as the most adept, with an 80%

accuracy in spatial comprehension tasks. OWL-ViT models paired with ReCLIP and GPT-4 achieved 40% and 50%, respectively, indicating moderate capabilities. Grounding DINO + ReCLIP followed closely with a 30% accuracy rate.

- **Superlatives:** When handling superlative expressions, Grounding DINO + GPT-4 again excelled with 80% accuracy. OWL-ViT + ReCLIP and OWL-ViT + GPT-4 both managed a lower but identical rate of 10%, whereas Grounding DINO + ReCLIP showed a modest improvement at 30%.
- **Multiple Proposals:** The Grounding DINO + GPT-4 configuration was the standout performer in scenarios with several valid object references, achieving 90% accuracy. In contrast, Grounding DINO + ReCLIP achieved a 70% success rate, and OWL-ViT models paired with ReCLIP and GPT-4 obtained 50% and 60%, respectively.
- **Self-referential:** In the category requiring an understanding of self-referential cues, Grounding DINO + GPT-4 maintained a strong performance with 60% accuracy. OWL-ViT + GPT-4 demonstrated improvement in this area with 20%, whereas the ReCLIP pairings for both OWL-ViT and Grounding DINO were aligned at 10% and 30%.
- **Household Objects:** A perfect score was attained by Grounding DINO + GPT-4, indicating its exceptional capability in recognizing everyday items, which is critical for practical applications. OWL-ViT + GPT-4 also performed well with 80%, suggesting good utility. Both ReCLIP pairings showed equal performance at 30%.
- **Overall Accuracy:** Aggregating the results across all tasks, Grounding DINO + GPT-4 dominated with an 80% overall accuracy. OWL-ViT + GPT-4 followed with 38.33%, showing that the combination with GPT-4 generally leads to better performance. The ReCLIP pairings, OWL-ViT + ReCLIP and Grounding DINO + ReCLIP, exhibited overall accuracies of 26.67% and 36.67%, respectively, indicating that these configurations might benefit from further optimization.

5.1 Analysis

The findings suggest that the integration of GPT-4 with either OWL-ViT or Grounding DINO significantly enhances performance across a variety of ReC tasks, with the Grounding DINO + GPT-4 pairing proving to be the most effective overall. This pairing demonstrates the potential for strong practical application, particularly in environments where accurate interpretation of complex queries is necessary.

It is important to know the impact that the performance of the zero-shot object detectors has on the entire model. It is crucial to recognize that the study evaluates the effectiveness of the complete detection and comprehension pipeline rather than the ReC models in isolation. Hence, the results could be influenced by the object detection performance of the zero-shot models. For instance, OWL-ViT exhibited limitations in object detection within the scene, which has downstream effects on task success. As seen in Figure 5.1, there are 3 birds in the scene correctly detected by Grounding DINO.

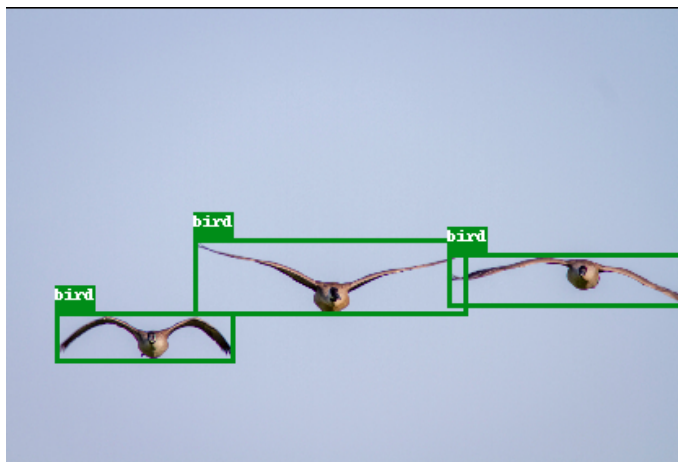


Figure 5.1: Grounding DINO successful detection

However, as seen in Figure 5.2, only 2 birds are detected. As the prompt was "second bird from the right" the expected output would be as shown in Figure 5.3. This would mean that using OWL-ViT for zero-shot object detection, either ReC methods would fail for this example. This discrepancy demonstrates that while GPT-4 shows promise as an adjunct or replacement for ReCLIP in ReC models, the choice of object detection model is a critical factor in the overall pipeline's performance.

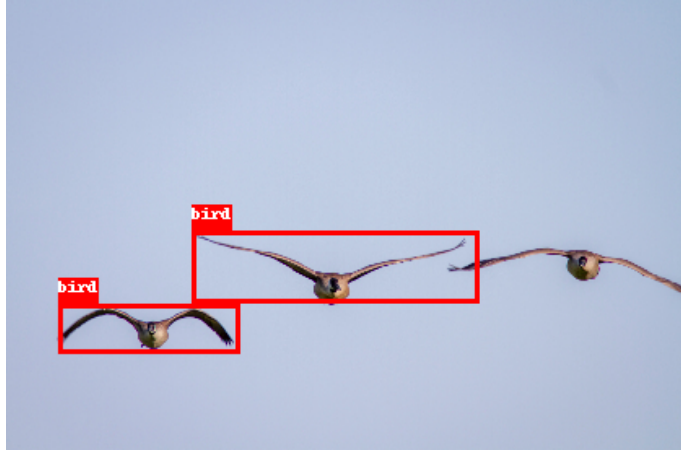
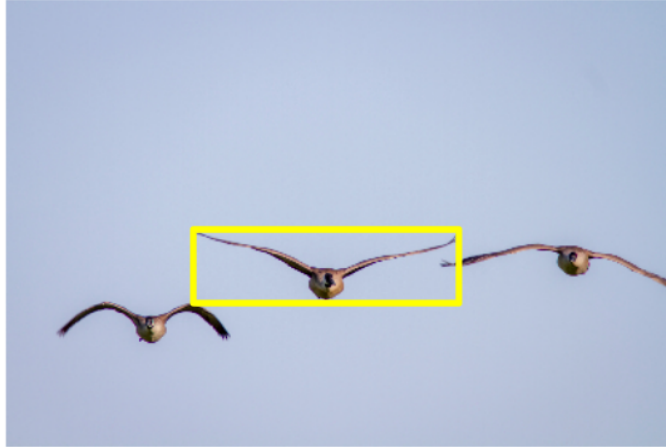


Figure 5.2: OWL-ViT failed detection



“Second bird from the right”

Figure 5.3: Expected ReC output

Another crucial criterion that was also identified to show the effectiveness of the model was whether the target objects in the image-text pair were identified. This is important as if no target objects were to be detected, the ReC component of the pipeline would be completely ineffective. Target objects were not detected in 36.67% of the dataset when using OWL-ViT, compared to 0% when using Grounding DINO. This value further supports our hypothesis that Grounding DINO is the better option to be used as the zero-shot object detector in our perception pipeline.

Figure 5.4 shows examples of results obtained from testing under the counting, spatial reasoning, and superlatives categories. The predictions made by the pipeline is marked in red bounding boxes and the ground truth is marked in the green bounding boxes. If the image contains only a red

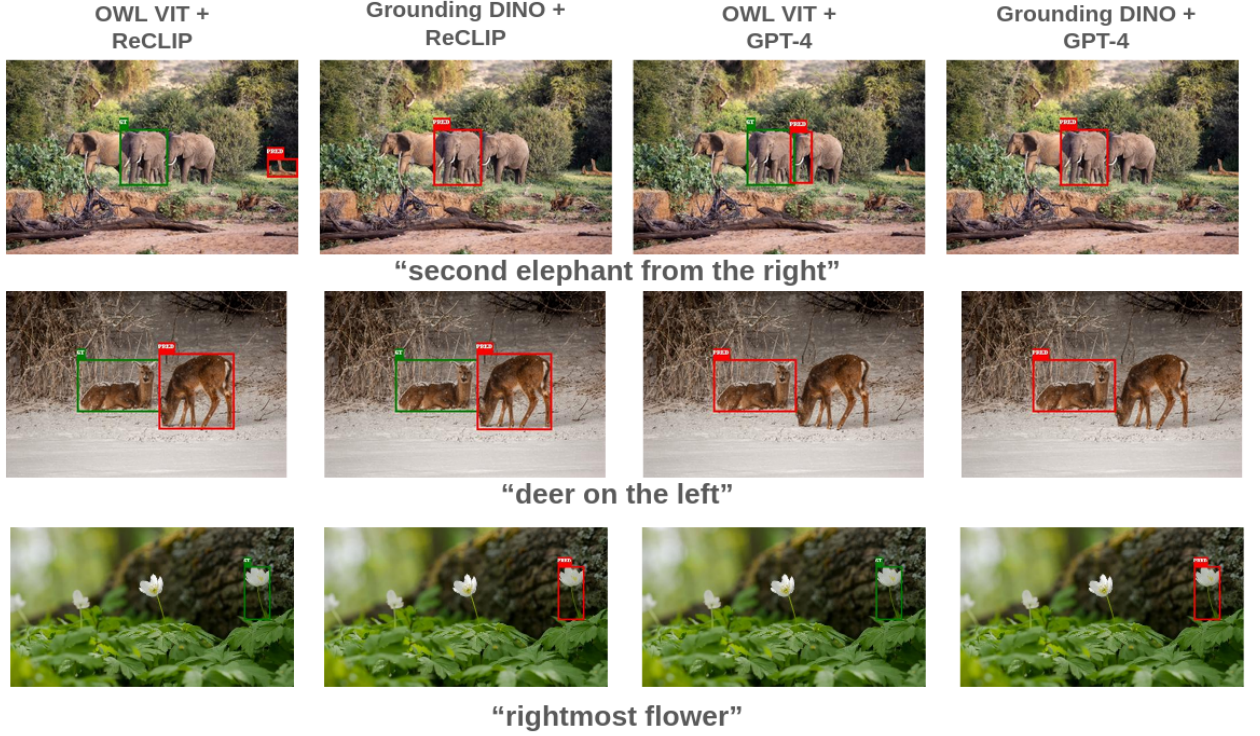


Figure 5.4: Examples of results for counting (top row), spatial reasoning (middle row), & superlatives (bottom row)

bounding box, it shows that the target object has been identified accurately. If the image contains only a green bounding box, it shows that a prediction was not made by the ReC model.

In the first row, a result under the counting category is shown. As seen from the results, pipelines containing Grounding DINO as the zero-shot object detectors succeeded in the ReC task. In the example of the result from pipelines using OWL-ViT as the zero-shot object detector, we see that the elephants are not detected accurately. This essentially led to the ReC model failing in the task.

In the second row, a result from the spatial reasoning category is shown. In this example, we are able to test the spatial reasoning abilities of the ReC models. This is possible because the target objects, deer, in the image have been accurately detected by the zero-shot object detectors. Through this test, we are able to see that pipelines containing ReCLIP failed this task unlike pipelines containing GPT-4 as the ReC model.

In the third row, a result from the superlatives category is shown. From this example, we are able to see that there has been no prediction made by models containing OWL-ViT. Due to

this, there had been no prediction made by the ReC models, as they would be unable to do so. Through this we are able to see the effect of the performance of the zero-shot object model on the performance of the entire pipeline.

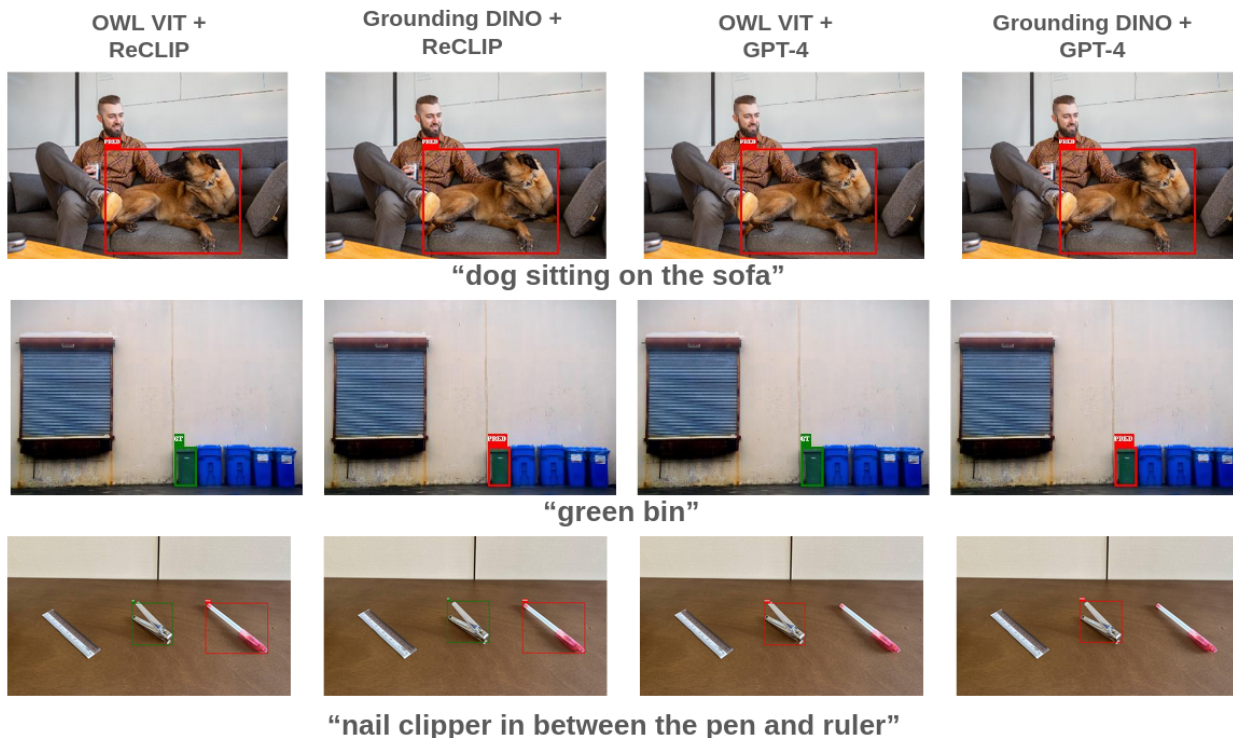


Figure 5.5: Examples of results for multiple instances (top row), self-referential (middle row), & household objects (bottom row)

Figure 5.5 shows examples of results from the multiple instances, self-referential, and household objects categories.

In the first row of images, we see a result from the multiple instances category. In this example, we are able to see that all four pipelines were able to predict the target object accurately. Generally, all four pipelines performed well in this category.

In the second row of images, an example from the self-referential category is shown. In this example, similar to the superlatives example shown in 5.4, pipelines containing OWL-ViT as the zero-shot object detectors did not make a prediction. This is due to OWL-ViT not detecting the target images in the image, in this case, bins. However, of the detected bins, using Grounding DINO, both ReCLIP and GPT-4 were able to detect the green bin accurately.

In the third row of images, results from the household objects category is shown. In this image,

we are able to see that the pipelines using GPT-4 as the ReC model were able to detect the target objects accurately. The pipelines containing ReCLIP as the ReC models were not able to accurately detect the target object. This is likely due to the poor spatial reasoning of the ReCLIP model.

Chapter 6

Conclusion and Future Work

The integration of VLMs for zero-shot object detection and ReC has substantially improved the robotic perception pipeline.

The improvements to the perception pipeline are:

1. Able to detect a broader array of objects without requiring prior training through zero-shot object detection
2. Able to disambiguate objects of the same class through the implementation of ReC

These improvements have ensured that the project objectives mentioned in Chapter 1 have been satisfied.

To conclude, GPT-4 performs well as an ReC model. It is important to note that although the Grounding DINO and GPT-4 combination performed the best among all combinations, it was mainly limited due to the performance of Grounding DINO on more complex images.

Future work will focus on working with better zero-shot object detectors, which would be able to detect objects in more complex situations. Another alternative would be to fine tune our zero-shot object detection models on a large dataset for our desired application. Through this, we can improve the performance of the zero-shot object detection model. The performance of ReC of the state of the art model, Re-CLIP, has also been improved by using GPT-4.

References

- [1] J. Sinapov, T. Bergquist, C. Schenck, U. Ohiri, S. Griffith, and A. Stoytchev, “Interactive object recognition using proprioceptive and auditory feedback,” *The International Journal of Robotics Research*, vol. 30, Sep. 2011. DOI: 10.1177/0278364911408368
- [2] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, “Object detection with deep learning: A review,” 2019. DOI: 10.48550/arXiv.1807.05511
- [3] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” 2019. DOI: 10.48550/arXiv.1506.01497
- [4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” 2016. DOI: 10.48550/arXiv.1506.02640
- [5] W. Liu et al., “Ssd: Single shot multibox detector,” in *Conference on Electronic Speech Signal Processing (ESSV)*, 2016. DOI: 10.48550/arXiv.1512.02325
- [6] A. Bansal, K. Sikka, G. Sharma, R. Chellappa, and A. Divakaran, “Zero-shot object detection,” in *European Conference on Computer Vision (ECCV)*, 2018. DOI: 10.48550/arxiv.1804.04340
- [7] P. F. Felzenszwalb, R. B. Girshick, D. Mcallester, and D. Ramanan, “Object detection with discriminatively trained part-based models,” in *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, 2010, p. 1627. DOI: 10.1109/TPAMI.2009.167
- [8] A. Coskun, “Object detection in 2022: The definitive guide,” 2022. [Online]. Available: <https://viso.ai/deep-learning/object-detection/>
- [9] L. Du, R. Zhang, and X. Wang, “Overview of two-stage object detection algorithms,” in *Journal of Physics: Conference Series (JPCS)*, vol. 1544, 2020. DOI: 10.1088/1742-6596/1544/1/012033

- [10] Q. Fan, L. Brown, and J. Smith, “A closer look at faster r- cnn for vehicle detection,” in *2016 IEEE intelligent vehicles symposium (IV)*, 2016, pp. 124–129. DOI: 10.1109/IVS.2016.7535375
- [11] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *IEEE conference on computer vision and pattern recognition (CVPR)*, 2017, pp. 2117–2125. DOI: 10.1109/CVPR.2017.106
- [12] Z. Cai and N. Vasconcelos, “Cascade r-cnn: High quality object detection and instance segmentation,” in *IEEE transactions on pattern analysis and machine intelligence*, vol. 43, 2019, pp. 1483–1498. DOI: 10.48550/arXiv.1906.09756
- [13] T. Sharma, B. Debaque, N. Duclos, A. Chehri, B. Kinder, and P. Fortier, “Deep learning-based object detection and scene perception under bad weather conditions,” in *Electronics 2022*, 2022. DOI: 10.3390/electronics11040563
- [14] A. Radford et al., “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, 2021. DOI: 10.48550/arXiv.2103.00020
- [15] M. Minderer et al., “Simple open-vocabulary object detection with vision transformers,” in *European Conference on Computer Vision (ECCV)*, 2022. DOI: 10.48550/arxiv.2205.06230
- [16] S. Liu et al., “Grounding dino: Marrying dino with grounded pre-training for open-set object detection,” 2023. DOI: 10.48550/arxiv.2303.05499
- [17] M. Caron et al., “Emerging properties in self-supervised vision transformers,” 2021. DOI: 10.48550/arXiv.2104.14294
- [18] L. H. Li et al., “Grounded language-image pre-training,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. DOI: 10.48550/arXiv.2112.03857
- [19] Z. Liu et al., “Swin transformer: Hierarchical vision transformer using shifted windows,” 2021. DOI: 10.48550/arXiv.2103.14030

- [20] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” 2019. DOI: 10.48550/arXiv.1810.04805
- [21] Y. Yao, A. Zhang, Z. Zhang, Z. Liu, T.-S. Chua, and M. Sun, “Cpt: Colorful prompt tuning for pre-trained vision-language models,” 2021. DOI: 10.48550/arXiv.2109.11797
- [22] P. Zhang et al., “Cpt: Colorful prompt tuning for pre-trained vision-language models,” in *IEEE/CVF International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 5575–5584. DOI: 10.48550/arXiv.2109.11797
- [23] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-cam: Visual explanations from deep networks via gradient-based localization,” in *International Journal of Computer Vision (IJCV)*, 2019. DOI: 10.48550/arXiv.1610.02391
- [24] S. Subramanian, W. Merrill, T. Darrell, M. Gardner, S. Singh, and A. Rohrbach, “Reclip: A strong zero-shot baseline for referring expression comprehension,” in *Association for Computational Linguistics (ACL)*, 2022. DOI: 10.48550/arxiv.2204.05991
- [25] Z. Yang et al., “The dawn of lmms: Preliminary explorations with gpt-4v(ision),” 2023. DOI: 10.48550/arXiv.2309.17421
- [26] X. Sui, S. Li, H. Yang, H. Zhu, and Y. Wu, “Language models can do zero-shot visual referring expression comprehension,” in *International Conference on Learning Representations (ICLR)*, 2023. [Online]. Available: <https://openreview.net/forum?id=F7mdgA7c2zD>
- [27] A. Chiche and B. Yitagesu, “Part of speech tagging: A systematic review of deep learning and machine learning approaches,” in *Journal of Big Data*, vol. 9, 2022. DOI: 10.1186/s40537-022-00561-y
- [28] M. Honnibal and I. Montani, “spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing,” To appear, 2017.
- [29] S. Bird, E. Klein, and E. Loper, *Natural language processing with Python: analyzing text with the natural language toolkit.* ” O’Reilly Media, Inc.”, 2009.

- [30] S. Kazemzadeh, V. Ordonez, M. Matten, and T. Berg, “Referitgame: Referring to objects in photographs of natural scenes,” in *Association for Computational Linguistics (ACL)*, 2014, pp. 787–798. DOI: 10.3115/v1/D14-1086